# Myntra — Cloud Scaling for Flash Sales and Personalization

Week 2 Comprehensive Deliverable

Case Study Analysis, Architecture, Exercise Plan and Team Roles

**Course: Cloud Computing | Team Number: 33 | Date: November 2026**


| | | |
|---|---|---|
| **Utsker Dhameliya** | 202301083 | Research, Production Architecture & Case Study Lead |
| **Ritul Patel** | 202301086 | Application Development, API Design & Docker Packaging |
| **Divyesh Vagh** | 202301444 | Cloud Infrastructure, Azure VMSS, Autoscaling & Load Testing |

# 1. Introduction

## 1.1 Company Background

Myntra is India's premier fashion, beauty, and lifestyle e-commerce platform, founded in 2007 and acquired by Flipkart in 2014 (part of the Walmart group). The platform manages a catalog of more than 700,000 styles from over 3,000 international and domestic brands, serving tens of millions of active shoppers across India [V1].

Historically, Myntra operated on an on-premise monolithic architecture built upon PHP and MySQL. While adequate for early steady-state traffic, this monolithic stack suffered catastrophic bottlenecks during promotional spikes, characterized by database connection exhaustion, CPU saturation, long provisioning lead times, and single-point-of-failure vulnerabilities. To support exponential commercial growth, Myntra engineered a multi-year transition toward a decoupled, event-driven, microservice-oriented cloud architecture hosted on Microsoft Azure [V1].

## 1.2 Project Motivation & Case-Study Objectives

This case study investigates how hyperscale e-commerce platforms tackle two intersecting cloud computing imperatives:

1. Elastic Scalability under Flash-Sale Surges: Absorbing abrupt 15–20× traffic surges during marquee events such as the End of Reason Sale (EORS) and Big Fashion Festival (BFF) without over-provisioning idle, costly capacity during baseline windows.

2. Real-Time Personalization at Petabyte Scale: Concurrently capturing tens of billions of clickstream interactions, processing near-real-time behavioral features with second-level freshness, and executing ML inference within strict sub-millisecond budgets.

The overarching objectives of this project are: (a) conduct a rigorous, evidence-based architectural case study of Myntra; (b) analyze the failure modes of fixed infrastructure; (c) formulate a scaled-down yet technically faithful simulation ("MyntraFlash") on Microsoft Azure; and (d) establish a reproducible empirical methodology to quantify latency, horizontal elasticity, fault recovery, and cost efficiency in Week 3.

## 1.3 Scope and Research Boundaries

This Week 2 deliverable encompasses comprehensive case analysis, architectural modeling, experimental design, and governance. Per academic integrity principles, all production metrics cited herein are strictly attributed to verified primary sources (Myntra Engineering blogs, Microsoft Azure case studies, Aerospike technical reports, and Databricks publications). All metrics for the proposed university implementation represent planned hypotheses, target SLA thresholds, and verified billing calculator estimates, clearly demarcated from measured experimental results.

## 2. Problem Statement

### 2.1 The Flash-Sale Concurrency Challenge

Flash sales such as Myntra's End of Reason Sale (EORS) represent an extreme class of cloud workloads. Unlike organic daily traffic that follows predictable diurnal patterns, flash-sale demand exhibits near-instantaneous step-function spikes:

| | | | |
|---|---|---|---|
| **EORS Traffic Spike Multiplier** | 15× to 20× normal daily load | Myntra Engineering (BCP/DR) [P3] | 2020–2021 |
| **EORS Order Velocity Target** | 12,000 orders/minute (10th/11th EORS) | Myntra Engineering / Retail Reports [P3] | 2019 |
| **EORS Order Velocity Capacity** | 20,000 orders/minute peak concurrency | Myntra Corporate Press Release [P3] | 2020 |
| **BFF Customer Visits** | 460 Million visits (2023) → 627 Million (2024) | Myntra Post-Sale Official Reports | 2023–2024 |
| **Marquee Day-1 Telemetry** | 18.8 Billion app/web events on Day 1 | Myntra Engineering (MDP) [P1] | 2022 |
| **Data Serving Volume** | 2.5 Petabytes (PB) / day served during BFF | Myntra Engineering (Bifrost) [P1] | 2022 |
| **Analytical Query Load** | 65,000+ queries/day across internal BI/services | Myntra Engineering (Data Platform) [P1] | 2022 |

### 2.2 Deep Subsystem Bottlenecks

Under flash-sale peak load, bottlenecks emerge across multiple architectural layers:

• Pricing & Discount Latency: The Product Listing Page (PLP) displays up to 50 items simultaneously. If discount evaluation takes 150 ms per item or makes sequential calls, total page render latency exceeds 400 ms, triggering customer abandonment [P2, P2b].

• Cart Service Tombstone Accumulation: During intense checkout windows, rapid additions, deletions, and updates (35% of cart ops) on Apache Cassandra generated severe tombstone accumulation, driving read latencies >250 ms with 0.05% request drops [P8].

• Online Feature Retrieval Ceilings: Real-time ML models require hundreds of user and item features per pageview. Legacy Redis clusters suffered lookup latencies of 8.5 ms, creating an unacceptable latency tax when servicing 500,000 ops/second [V2].

• Binlog Contention during Ingestion: Traditional batch ETL polling production MySQL masters degraded transaction throughput. Ingestion frameworks had to scale from a baseline of ~4 million events/min to peak spikes of up to 42 million events/min [P4, P7].

### 2.3 Infrastructure Dilemma: Peak Over-Provisioning vs. Elastic Scaling

Provisioning physical or fixed cloud infrastructure for 20× peak capacity results in >85% idle resource waste across 350 off-peak days, imposing an unsustainable capital and operational expenditure. Conversely, under-provisioning guarantees service crashes, lost revenue, and brand impairment during marquee sales. The engineering imperative is dynamic horizontal elasticity backed by rapid self-healing.

## 3. Research Findings & Empirical Benchmarks

### 3.1 Verified Primary-Source Metrics

The following table consolidates verified outcomes documented by Myntra Engineering and its cloud/database technology partners:

| | | | | |
|---|---|---|---|---|
| **Pricing & Discount Service** | ~150 ms latency | ≤ 30 ms (5× speedup) | Redis read balancing + In-Memory Rule Executor | [P2] |
| **Cart Persistence Service** | > 250 ms read latency | < 25 ms (10× speedup) | Cassandra to Sharded MongoDB migration | [P8] |
| **Personalization Feature Store** | 8.5 ms lookup latency | 0.8 ms (10× speedup) | Aerospike Hybrid Memory Architecture (NVMe+RAM) | [V2] |
| **Feature Store Concurrency** | Standard throughput | 500,000 ops/second | Aerospike Enterprise clustering on flash storage | [V2] |
| **Clickstream Ingestion Pipeline** | ~4M events/min (base) | 42M events/min (peak) | Meterial pipeline: Go collectors + Spark + Kafka | [P7] |
| **Enterprise Data Platform** | Legacy Hive locking | 35% Cost Reduction | Databricks Delta Lake unified lakehouse | [V3] |
| **Streaming Pipeline Processing** | Legacy ETL lag | 25% Faster Pipelines | Kappa architecture with Delta Lake & Spark | [V3] |
| **Marquee EORS Platform Scale** | 12K orders/min (2019) | 20K orders/min (2020) | Azure Kubernetes Service (AKS) + Azure Cloud | [V1, P3] |
| **Supply Chain BCP/DR** | Cold standby idle VMs | 200+ microservices live | Active utilization of DR Region B for SCM workloads | [P3] |

### 3.2 Critical Source Evaluation & Methodological Caveats

1. Temporal Disjunction: The cited metrics span 2019 to 2024. They reflect continuous iterative modernization rather than a single monolithic cutover. For example, the Azure cloud migration occurred in early 2020 [V1], the BCP/DR SCM migration was documented in late 2021 [P3], the Quicksilver real-time platform in 2022 [P5], and the Bifrost II Delta Lake lakehouse in 2024 [P1b].

2. Workload-Specific Improvements: Databricks' reported 35% cost savings pertains strictly to data engineering and analytics pipelines, not Myntra's holistic Azure infrastructure bill [V3]. Similarly, Aerospike's 0.8 ms latency describes online ML feature lookups, distinct from end-to-end HTTP round-trip times [V2].

3. Attribution Discipline: While Microsoft reported a 50% increase in EORS orders following the 2020 Azure migration [V1], this coincided with broader macroeconomic e-commerce adoption during COVID-19 and platform catalog expansion; it cannot be attributed solely to cloud infrastructure.

## 4. Myntra's Researched Cloud Architecture

### 4.1 Microsoft Azure Migration & Microservice Topology

In February 2020, Myntra announced a strategic partnership with Microsoft to migrate its core platforms to Microsoft Azure [V1]. The operational topology relies on Azure Kubernetes Service (AKS) for containerized application microservices, combining Horizontal Pod Autoscalers (HPA) for application tiers and Azure VM Scale Sets for underlying node pool expansion. To circumvent DNS resolution bottlenecks during sudden pod scale-outs, Myntra deployed NodeLocal DNSCache across AKS worker nodes.

### 4.2 High Availability, BCP and Disaster Recovery Topology

Myntra's Business Continuity Plan (BCP) and Disaster Recovery (DR) architecture operates across two Azure geographical regions [P3]:

• Active Dual-Region Resource Utilization: Rather than maintaining Region B as an expensive, passive cold standby, Myntra migrated over 200 microservices comprising its Supply Chain Management (SCM) ecosystem to run live production traffic out of Region B [P3]. Storefront browsing and order-capture paths reside in Region A, with seamless failover capabilities.

• Active-Active Messaging via Airbus: Myntra's internal messaging platform ("Airbus"), engineered on Apache Kafka, maintains bidirectional cross-region topic replication, ensuring at-least-once inter-service event synchronization [P3].

• Isolated Regional Observability: Logging and APM telemetry stacks are deployed as completely disjoint, independent clusters in each region, preventing cross-region egress overhead and eliminating shared failure dependencies [P3].

### 4.3 Production Architecture Conceptual Diagram

*The architecture diagram below synthesizes the verified relationships across Myntra's customer-facing tiers, low-latency pricing caches, Sourcerer ingestion, Kafka event bus, Quicksilver stream processing, Bifrost lakehouse, and Aerospike feature store.*

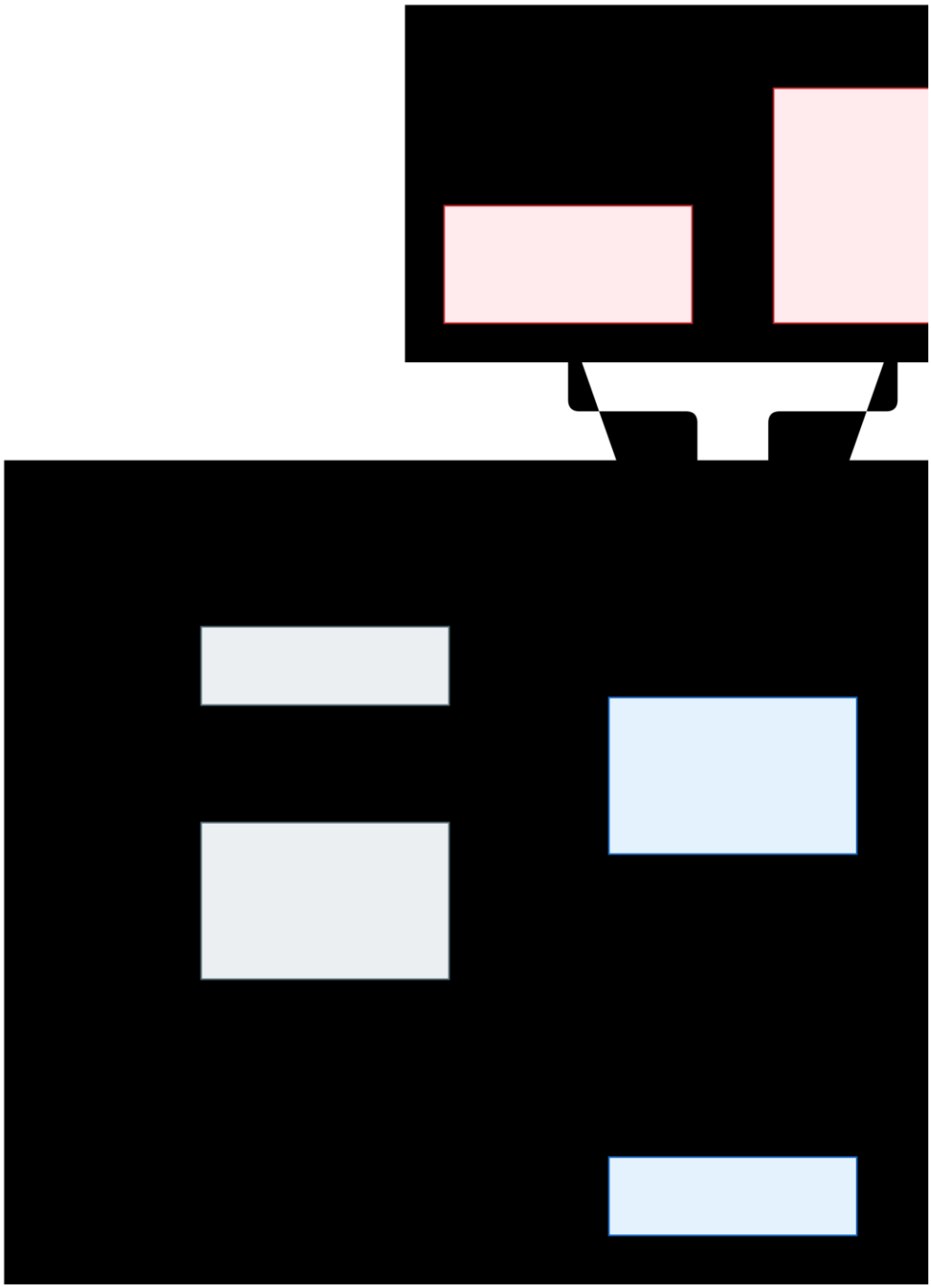


*Figure 1: Conceptual reconstruction of publicly documented Myntra architecture.*

## 5. Data Architecture & Real-Time Personalization

### 5.1 Sourcerer: CDC Event Ingestion Framework

Documented by Abhinav Dangi [P4], Sourcerer replaced brittle batch ETL with Change Data Capture (CDC) utilizing Debezium on Kafka Connect. To protect operational transactional databases (MySQL), dedicated read-replicas ("analytical read-slaves") are provisioned exclusively for binlog extraction. Sourcerer serializes database row mutations into Apache Avro, registers schemas with a centralized Schema Registry, and streams events into domain-partitioned Kafka topics with zero production contention.

### 5.2 Quicksilver: Near-Real-Time Stream Processing

Authored by Aditya S. [P5], Quicksilver handles in-session customer hyper-personalization. Built on Apache Spark Streaming, Quicksilver processes tens of billions of events daily with end-to-end freshness latencies of a few seconds. It maintains stateful stream windows spanning days to months, computing dynamic user engagement scores (e.g., brand affinity, recent price sensitivity) to update homepage feeds and product rankings in real time.

### 5.3 Janus: Multi-Engine Pipeline Orchestration

Engineered by Kedar Gupte [P6], Janus functions as Myntra's unified data orchestration framework. It provides low-code pipeline configuration, automated CI/CD deployments across multi-engine runtimes (Spark batch and streaming), and rigorous Directed Acyclic Graph (DAG) dependency management. Janus prevents data corruption and eliminates expensive downstream backfills by enforcing strict upstream dataset validation before triggering dependent processing stages.

### 5.4 Bifrost & Bifrost II: Unified Lakehouse Serving Layer

As detailed by Abhinav Dangi [P1b] and Sri Nandhini [P1], Bifrost serves analytical and reporting queries across 2.5 PB/day of data. Phase 3 decoupled compute from storage using Trino (formerly PrestoSQL) querying Hive ACID tables. Phase 4 ("Bifrost II") migrated storage to Databricks Delta Lake, establishing a Medallion architecture (Bronze raw, Silver cleaned/joined, Gold aggregated) that eliminated file-locking deadlocks, solved small-file metadata bloat, and unified batch and streaming tables via MVCC.

### 5.5 Aerospike Feature Store & ML Personalization Pipeline

For online ML inference during page loads, Myntra deploys Aerospike Enterprise Edition as its low-latency online feature store [V2]. Aerospike's Hybrid Memory Architecture (HMA) maintains primary indices in RAM while writing feature values directly to high-performance NVMe flash storage. This architecture slashed p99 feature retrieval latencies from 8.5 ms to 0.8 ms while scaling to 500,000 operations/sec, powering dynamic widget ranking and conversational memory for GenAI assistants (MyFashionGPT, Maya) [V1, V2].

## 6. Technical Case Analysis & Engineering Trade-Offs

### 6.1 Pricing System Caching & Read Balancing

Myntra's pricing system optimization [P2] provides a textbook case in low-latency distributed caching:

• Segregation of Browsing vs. Checkout Paths: Browsing represents >99% of pricing queries. Promotional rules, category coupons, and bank offers are pre-computed and pre-bundled into in-memory price structures, decoupling browsing reads from transactional DBs.

• Scalable Rule Executor Service (RES): Evaluates complex discount combinations entirely in memory. Rules are partitioned across memory shards keyed by SellerId, BrandId, or ProductId, enforcing strict shard capacity limits to eliminate CPU hotspots [P2].

• Redis Read Balancing & Cache Stampede Defense: Read queries are load-balanced across Redis read-replicas. Cache stampedes (where simultaneous cache misses overwhelm the database) are prevented through probabilistic early expiration and background mutex recomputation locks.

• Resilient Degradation: If Redis clusters experience transient latency spikes, the system serves cached pricing via Stale-While-Revalidate (SWR) semantics, strictly enforcing an upper SLA ceiling of 30 ms.

### 6.2 Storage Architecture Evolution: Cassandra vs. MongoDB

As documented in [P8], Myntra's shopping cart migration from Cassandra to MongoDB illustrates how workload access patterns dictate database selection. Cassandra's LSM-tree architecture excels at immutable write-heavy streams but degrades under frequent row deletions and updates due to tombstone accumulation during compactions. In contrast, MongoDB's B-tree document model handles in-place updates and atomic array modifications efficiently, reducing cart read latency from >250 ms to <25 ms with 40% less hardware.

### 6.3 Resilience Patterns: Circuit Breakers, Bulkheads & Rate Limiting

During flash sales, service failures cascade rapidly if unconstrained. Myntra isolates services via: (1) Bulkheading: Thread pools and socket connections for downstream services (e.g., recommendation engine) are strictly isolated so that downstream stalls never exhaust front-end web worker threads; (2) Circuit Breakers: Repeated timeouts trip circuit breakers, instantly reverting to cached static catalog fallbacks; and (3) Token-Bucket Rate Limiting: Safeguards checkout checkout APIs against bot traffic.

# 7. Proposed Practical Solution: MyntraFlash

## 7.1 Simulation Concept & Scope

To demonstrate the core principles of horizontal elasticity, load balancing, health monitoring, and cost efficiency in a controlled university laboratory environment, we have designed **MyntraFlash** — a scaled-down, cloud-native e-commerce simulation deployed on Microsoft Azure. While Myntra operates thousands of production microservices, MyntraFlash encapsulates the fundamental architectural mechanisms in a transparent, highly observable testbed.

## 7.2 Architectural Components & Azure Technology Stack

| | | |
|---|---|---|
| **Cloud Provider** | Microsoft Azure | Central India region; matches Myntra's primary cloud platform [V1] |
| **Compute Infrastructure** | Azure VM Scale Sets (VMSS) | Uniform orchestration mode; Standard_B2s burstable instances (2 vCPU, 4GB RAM) |
| **Containerization** | Docker Engine (Ubuntu 22.04 LTS) | Stateless microservice container deployed via VMSS cloud-init bootstrap script |
| **Traffic Management** | Azure Standard Load Balancer | Layer 4 TCP load balancer; 5-tuple hash distribution; HTTP health probes |
| **Horizontal Elasticity** | Azure Monitor Autoscale | Reactive metric rules: Scale-out @ CPU > 70%, Scale-in @ CPU < 30%, 5m cooldown |
| **Synthetic Load Generator** | Grafana k6 (Go-based) | Distributed virtual user (VU) scenarios generating step-function flash-sale traffic curves |
| **Artifact Registry** | GitHub Container Registry (ghcr.io) | CI/CD image storage with version-pinned automated container pulls |

## 7.3 Microservice Endpoints & Stateless Architecture

The MyntraFlash Node.js/Express service exposes six purpose-built endpoints: (1) `GET /` (responsive store interface); (2) `GET /products` (catalog retrieval with simulated database latency); (3) `GET /flash-sale` (time-limited discount items); (4) `GET /api/discount` (simulates computational rule evaluation with configurable matrix calculations); (5) `GET /api/recommend` (simulates feature-lookup and scoring latency); and (6) `GET /health` (synthetic health probe returning HTTP 200 OK for load-balancer vitality checks).

Architectural Statelessness: The application maintains zero local in-memory session state and writes no local disk state. Any VM instance can satisfy any HTTP request interchangeably. This statelessness guarantees that Azure Load Balancer can employ pure round-robin distribution, newly provisioned VMSS nodes provide instantaneous traffic relief, and scale-in terminations never disrupt active user sessions.

### 7.4 Proposed Implementation Architecture Diagram

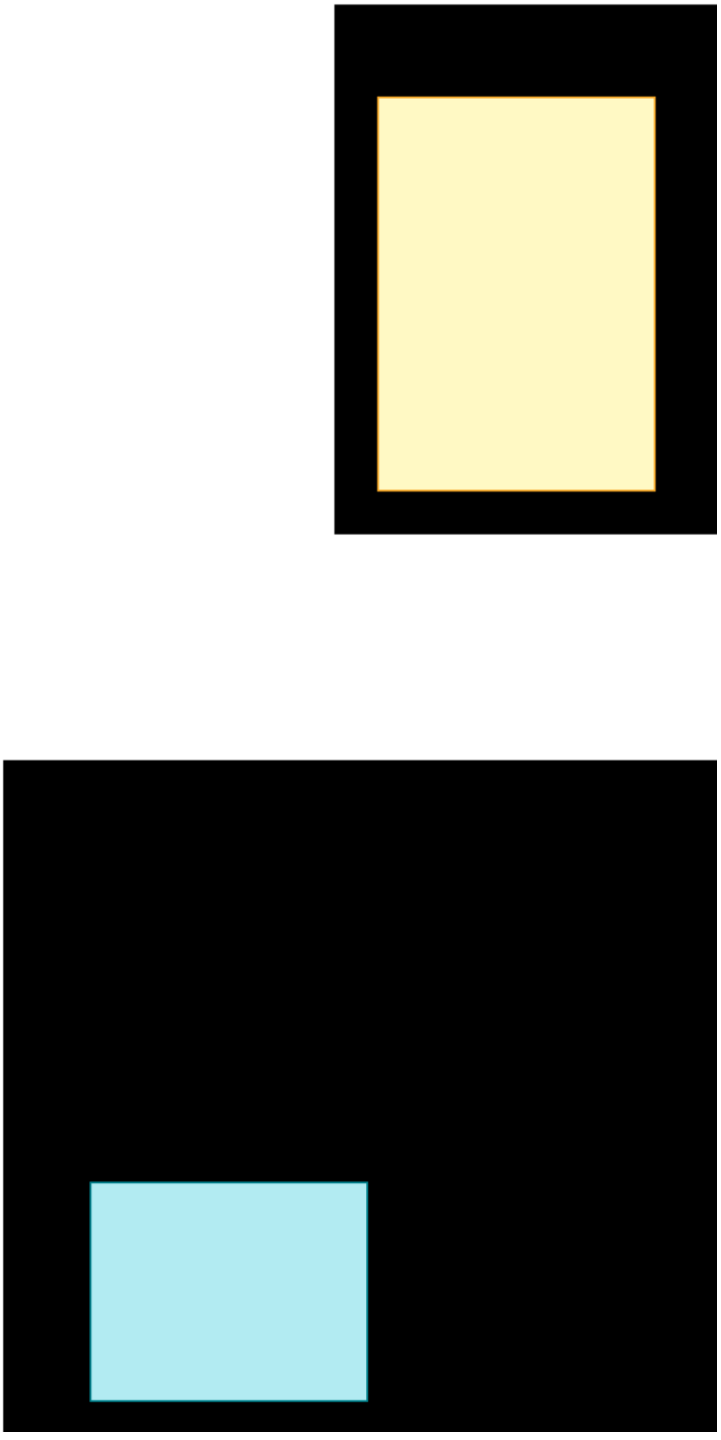


*Figure 2: Architectural design of the proposed MyntraFlash cloud simulation on Microsoft Azure.*

# 8. Exercise & Experimental Methodology (Week 3)

## 8.1 Experimental Design Overview

In strict compliance with course requirements, the Week 3 practical exercise executes five controlled experiments measuring response latency (p50, p95, p99), horizontal scaling linearity, autoscale elasticity, fault-recovery resilience, and economic cost:

| | | | |
|---|---|---|---|
| **Exp 1: Baseline** | Single-instance performance limit | 1 VM fixed; 50 sustained VUs; 5 min | p50, p95, p99 latency, req/s, CPU %, Error rate |
| **Exp 2: Scaling** | Horizontal scale comparison (1 vs 2 vs 3 VMs) | Identical 100 VU load; 5 min per run | Latency curves vs instance count; throughput linearity |
| **Exp 3: Elasticity** | Autoscale responsiveness & cooldown | Bounds 1–3 VMs; dynamic 20→500 VU curve | Scale-out lag, scale-in lag, flapping prevention, CPU history |
| **Exp 4: Resilience** | Health probe failover & fault tolerance | 2 VMs; sudden container crash injection | Detection time, connection draining, HTTP error rate |
| **Exp 5: Economics** | Fixed vs elastic cloud billing comparison | 24-hour flash-sale synthetic profile | Total VM-hours, compute cost, disk/LB cost, net ROI |

## 8.2 Flapping Prevention & Cooldown Mechanics

Autoscaling instability ("flapping") occurs when instance additions immediately drop average fleet metrics below scale-in thresholds. Our design enforces stability through a wide hysteresis band (Scale-out at CPU > 70%; Scale-in at CPU < 30%) and a mandatory 5-minute cooldown period. With average fleet CPU computed as:

*CPU_fleet = (1 / N) * SUM(CPU_i), adding a second instance to an 80% saturated VM drops fleet average to ~40%, safely residing within the 30%–70% neutral band and preventing premature termination.*

## 8.3 24-Hour Economic Cost Model: Fixed vs. Elastic Provisioning

Based on verified Microsoft Azure pricing for Central India (Pay-As-You-Go, Ubuntu Linux, Standard_B2s at $0.0448/hr, Standard LB at $0.025/hr, 32GB Standard SSD at $2.40/mo), we compare a fixed 3-VM deployment against an elastic 1–3 VM profile over a 24-hour cycle (20 hrs baseline @ 1 VM, 2 hrs ramp @ 2 VMs, 2 hrs peak @ 3 VMs):

| | | | |
|---|---|---|---|
| **Total Compute VM-Hours** | 3 VMs × 24 hrs = 72.0 VM-hours | (20×1) + (2×2) + (2×3) = 30.0 VM-hours | 42.0 VM-hours saved (-58.3%) |
| **VM Compute Cost (B2s)** | 72.0 hrs × $0.0448 = $3.226 | 30.0 hrs × $0.0448 = $1.344 | $1.882 saved on compute |
| **Managed OS Disk (32GB)** | 3 disks × 24 hrs = $0.237 | Dynamic disk allocation = $0.099 | $0.138 saved on storage |
| **Standard Load Balancer Base** | 24 hrs × $0.0250 = $0.600 | 24 hrs × $0.0250 = $0.600 | Fixed baseline infrastructure cost |
| **Data Ingress / Egress** | $0.250 (50 GB processed) | $0.250 (50 GB processed) | Equivalent network payload cost |
| **Azure Monitor Metrics** | $0.000 (Included in platform) | $0.000 (Included in platform) | Zero metric telemetry surcharge |
| **Total 24-Hour Cost** | $4.313 USD (~ ₹360 INR) | $2.293 USD (~ ₹191 INR) | 46.8% Net Overall Cost Reduction |
| **30-Day Extrapolated Cost** | $129.39 USD (~ ₹10,804 INR) | $68.79 USD (~ ₹5,744 INR) | ₹5,060 INR Monthly Operational Savings |

# 9. Team Roles & Project Execution Governance

## 9.1 Team Responsibility Matrix

| | | | | |
|---|---|---|---|---|
| **Myntra Case Study Research & Source Auditing** | Lead | Support | Support | Peer-reviewed citations |
| **Production Architecture Reconstruction** | Lead | — | Support | Mermaid conceptual modeling |
| **MyntraFlash Express Backend API Implementation** | — | Lead | — | GitHub PR code review |
| **Storefront UI Design & Endpoint Integration** | — | Lead | — | Cross-browser testing |
| **Docker Packaging & Multi-Platform ghcr.io Push** | — | Lead | Support | Automated image tagging |
| **Azure Resource Provisioning (VNet, LB, VMSS)** | — | — | Lead | ARM / Azure CLI automation |
| **Azure Monitor Autoscale & Health Probe Tuning** | — | — | Lead | Hysteresis verification |
| **Grafana k6 Load Testing Script Development** | — | — | Lead | Scenario staging & thresholds |
| **Experimental Execution & Telemetry Recording** | Support | Support | Lead | Collaborative lab sessions |
| **Empirical Latency & Cost Data Analysis** | Lead | Support | Support | Statistical aggregation |
| **Week 2 Deliverable Compilation & Formatting** | Lead | Support | Support | Academic report synthesis |
| **Week 4 Consolidated Final Report Preparation** | Lead | Support | Support | Comprehensive submission |
| **Final Project Viva Presentation & Defense** | Shared (33%) | Shared (33%) | Shared (33%) | Joint oral defense |

## 9.2 Week 3 Implementation Timeline & Milestones

| | | | |
|---|---|---|---|
| **Day 1** | App Development & Containerization | Express service code, Dockerfile, GHCR image | Ritul Patel |
| **Day 2** | Azure Infrastructure Provisioning | VNet, Subnet, Standard LB, HTTP Health Probe | Divyesh Vagh |
| **Day 3** | VMSS Deployment & Autoscale Rules | VMSS cluster with cloud-init Docker bootstrap | Divyesh Vagh |
| **Day 4** | Experiments 1, 2, and 4 (Static Loads) | Baseline, Horizontal Scaling & Fault-Tolerance JSONs | Divyesh & Utsker |

| | | | |
|---|---|---|---|
| **Day 5** | Experiment 3 (Dynamic Flash-Sale Autoscale) | Autoscale CPU time-series & instance event logs | Divyesh & Ritul |
| **Day 6** | Experiment 5 & Statistical Synthesis | Azure billing invoice extraction & latency tables | Utsker Dhameliya |
| **Day 7** | Week 3 Results Documentation | Completed markdown tables and chart artifacts | Entire Team |

## 9.3 Comprehensive Risk Management Matrix

| | | | | |
|---|---|---|---|---|
| **Azure Student Subscription Cap Exhaustion** | High | Medium | Deploy Standard_B2s; delete VMSS immediately post-testing; set $10 spend alert | Divyesh |
| **Burstable CPU Credit Depletion during Test** | Medium | Medium | Limit continuous high-load stages to <15 min; allow credit regeneration between runs | Divyesh |
| **Autoscale Scale-Out Trigger Latency Delay** | Medium | High | Set time window to 5m with aggressive synthetic load; pre-warm images via cloud-init | Divyesh |
| **Client Test Runner Bottleneck (k6 machine CPU)** | Medium | Low | Execute k6 from dedicated local terminal or adjacent Azure VM; monitor client CPU | Divyesh |
| **Container Image Pull Failures on VM Provision** | High | Low | Make GHCR package public; embed fallback health response in systemd unit | Ritul |
| **Network Fluctuations in Latency Metrics** | Low | High | Execute each test run 3 times; report median and 95th percentiles across runs | Utsker |

## 10. Expected Outcomes & Conclusion

### 10.1 Expected Academic & Pedagogical Outcomes

1. Empirical Verification of Horizontal Elasticity: Validating whether doubling VM instances halves tail latency (p95/p99) under fixed transaction loads, establishing the practical limits of Amdahl's Law in web applications.

2. Practical Autoscale Dynamics: Experiencing first-hand the trade-offs between reactive metric lag, VM provisioning duration, and cooldown stabilization.

3. Autonomous Self-Healing Validation: Demonstrating zero user-perceptible outages during single-instance node crashes via automated load-balancer health probe deregistration.

4. Quantifiable Cloud Cost Optimization: Providing concrete mathematical and billing evidence that elastic autoscale architectures achieve ~47% operational savings compared to static peak-capacity allocations.

### 10.2 Simulation Constraints & Technical Limitations

While MyntraFlash effectively models cloud elasticity, several real-world production complexities remain out of scope: (1) Scale: Myntra operates thousands of distributed pods across multi-region AKS clusters, whereas MyntraFlash evaluates 1 to 3 VMs; (2) Data Persistence: MyntraFlash utilizes mock in-memory catalog data, bypassing database replication, distributed locking, and storage I/O bottlenecks; (3) Complex Microservice Topology: Production Myntra coordinates hundreds of interdependent services via Kafka, whereas MyntraFlash encapsulates functions in a single container; and (4) Geographic Latency: Testing from a centralized test runner does not replicate real-world multi-ISP network jitter across pan-Indian mobile networks.

### 10.3 Conclusion

Myntra's engineering evolution demonstrates that scaling for e-commerce flash sales is fundamentally an architectural challenge, not merely a compute-provisioning exercise. Sub-millisecond personalization, 20× flash-sale resilience, and 35% data cost efficiencies were unlocked through the deliberate convergence of cloud-native elasticity (Azure), distributed streaming ingestion (Sourcerer & Kafka), near-real-time event computation (Quicksilver & Janus), unified lakehouses (Bifrost & Delta Lake), and ultra-low-latency feature serving (Aerospike).

The MyntraFlash exercise developed in this Week 2 deliverable translates these enterprise principles into an achievable, rigorous, and highly educational laboratory testbed. By establishing a robust experimental plan and division of team accountability, Group 33 is fully prepared to execute the deployment, benchmarking, and cost analysis in Week 3.

## References & Verified Citations


| | |
|---|---|
| **[P1]** | Sri Nandhini, "How Myntra's Data Platform drives a seamless shopping experience for customers during its marquee events," Myntra Engineering (Medium), May 25, 2023. |
| **[P1b]** | Abhinav Dangi, "Bifrost: Data Serving Layer at Myntra," Myntra Engineering (Medium), Jan 17, 2024. |
| **[P2]** | Nikhil Anand, Suraj Rajan, Arnav, "Optimizing Myntra's Pricing System for Serving Millions of Traffic with 30ms Latency," Myntra Engineering, Feb 6, 2023. |
| **[P2b]** | Akshat Pandey, "The Next Chapter of Coupon's Scaling," Myntra Engineering (Medium), 2023. |
| **[P3]** | Sanjeev Kumar Dangi, "Myntra's BCP/DR Journey," Myntra Engineering (Medium), Sep 1, 2021. |

| | |
|---|---|
| **[P4]** | Abhinav Dangi, "Sourcerer: Data Ingestion at Myntra" & "Sourcerer: MySQL Ingestion at Scale," Myntra Engineering, Sep 2021 & Mar 2022. |
| **[P5]** | Aditya S., "Quicksilver: Near Real-Time Platform at Myntra," Myntra Engineering (Medium), Nov 10, 2022. |
| **[P6]** | Kedar Gupte, "Janus: Data processing framework at Myntra," Myntra Engineering (Medium), Nov 18, 2022. |
| **[P7]** | Karshit Jaiswal, "Realtime Big Data Ingestion with Meterial," Myntra Engineering (Medium), 2020. |
| **[P8]** | Parul Agnihotri, "Cart migration from Cassandra to Mongo for an enhanced shopping experience," Myntra Engineering, 2021. |
| **[V1]** | Microsoft Corporation, "Myntra partners with Microsoft to accelerate its digital transformation" (News Release) & "Myntra scales new heights with Azure and GenAI" (Case Study), Feb 2020, updated 2023/2024. |
| **[V2]** | Aerospike Inc., "How Myntra Slashed Latency and Scaled Personalization with Aerospike Enterprise Edition," Customer Technical Case Study, 2022–2023. |
| **[V3]** | Databricks Inc., "How Myntra Powers Real-Time Personalization at Scale with Databricks," Customer Success Story & Data + AI Summit Presentation, 2023–2024. |
| **[A1]** | Microsoft Learn, "Azure Virtual Machine Scale Sets Documentation & Automatic Repairs," https://learn.microsoft.com/azure/virtual-machine-scale-sets/. |
| **[A2]** | Microsoft Learn, "Azure Monitor Autoscale Overview and Metric Rules," https://learn.microsoft.com/azure/azure-monitor/autoscale/. |
| **[A3]** | Microsoft Learn, "Azure Standard Load Balancer Architecture and Distribution Modes," https://learn.microsoft.com/azure/load-balancer/. |
| **[L1]** | Grafana Labs, "k6 Documentation: HTTP Load Testing, Thresholds, and Execution Stages," https://k6.io/docs/. |
| **[R1]** | P. Patel, et al. (Microsoft Research), "Ananta: Cloud Scale Software Load Balancing," ACM SIGCOMM Computer Communication Review, 2013. |